

Builder design pattern

16 September 2026 10:22

The **Builder Design Pattern** is a creational design pattern used when you need to create an object that has **many fields/options**, especially when many of them are **optional**.

1. What problem does Builder solve?

Suppose we have this class:

```
Java
class User {
    String name;
    int age;
    String email;
    String phone;
    String address;
    String city;
    String country;
}
```

We want to create a `User`.

One approach is a constructor:

```
Java
User user = new User(
    "Supraja",
    25,
    "abc@gmail.com",
    "9999999999",
    "Address",
    "Hyderabad",
    "India"
);
```

Problem?

You have to remember:

What does argument 4 mean?
What does argument 5 mean?

And what if some fields are optional?

```
Java
User user = new User(
    "Supraja",
    25,
    null,
    null,
    null,
    "Hyderabad",
    "India"
);
```

This becomes ugly very quickly.

2. Another solution: multiple constructors

You could create:

```
Java
User(String name)
User(String name, int age)
User(String name, int age, String email)
User(String name, int age, String email, String phone)
...
```

But now you have **constructor explosion**.

This is called the **telescoping constructor problem**.

The telescoping constructor problem is a coding anti-pattern that happens when a class has multiple overloaded constructors with an increasing number of parameter

3. Builder solves this

Instead, we create a Builder:

```

// Java

User user = new User.Builder()
    .name("Supraja")
    .age(25)
    .email("abc@gmail.com")
    .city("Hyderabad")
    .country("India")
    .build();

```

This is much easier to read.

Notice something important:

```

// Java

.name("Supraja")
.age(25)
.email("abc@gmail.com")
.city("Hyderabad")

```

You can immediately understand what each value represents.

4. How does Builder actually work?

Let's build it slowly.

Step 1 — Our actual object

```

// Java

class User {

    private String name;
    private int age;
    private String email;
    private String phone;
    private String city;

}

```

But instead of directly creating `User`, we create a `Builder`.

```

// Java

class User {

    private String name;
    private int age;
    private String email;
    private String phone;
    private String city;

    static class Builder {

        private String name;
        private int age;
        private String email;
        private String phone;
        private String city;

    }

}

```

Save your house.

5. Add methods to Builder

```

// Java

static class Builder {

    private String name;
    private int age;
    private String email;
    private String phone;
    private String city;

    public Builder name(String name) {
        this.name = name;
        return this;
    }

    public Builder age(int age) {
        this.age = age;
        return this;
    }

    public Builder email(String email) {
        this.email = email;
        return this;
    }

    public Builder phone(String phone) {
        this.phone = phone;
        return this;
    }

    public Builder city(String city) {
        this.city = city;
        return this;
    }

}

```



The important part is:

```
Java
return this;
```



Why?

Why?

Because it allows method chaining.

For example:

```
Java
new User.Builder()
    .name("Supraja")
    .age(25)
    .city("Hyderabad");
```



After:

```
Java
.name("Supraja")
```



the method returns the same Builder object.

So we can call:

```
Java
.age(25)
```



on it.

6. Finally, build the User

We add:

```
Java
public User build() {
    return new User(this);
}
```



And the User constructor:

```
Java
private User(Builder builder) {
    this.name = builder.name;
    this.age = builder.age;
    this.email = builder.email;
    this.phone = builder.phone;
    this.city = builder.city;
}
```



Now complete example:

```
class User {

    private String name;
    private int age;
    private String email;
    private String phone;
    private String city;

    private User(Builder builder) {
        this.name = builder.name;
        this.age = builder.age;
        this.email = builder.email;
        this.phone = builder.phone;
    }
}
```

```
        this.city = builder.city;
    }

    static class Builder {

        private String name;
        private int age;
        private String email;
        private String phone;
        private String city;

        public Builder name(String name) {
            this.name = name;
            return this;
        }

        public Builder age(int age) {
            this.age = age;
            return this;
        }

        public Builder email(String email) {
            this.email = email;
            return this;
        }

        public Builder phone(String phone) {
            this.phone = phone;
            return this;
        }

        public Builder city(String city) {
            this.city = city;
            return this;
        }

        public User build() {
            return new User(this);
        }
    }
}
```

Main method:

```
User user = new User.Builder()
    .name("Supraja")
    .age(25)
    .email("abc@gmail.com")
    .city("Hyderabad")
    .build();
```

7. Understand the flow

This is the most important part.

When you write:

```
// Java
User user = new User.Builder()
    .name("Supraja")
```

```
.age(25)
.city("Hyderabad")
.build();
```

Think:

Step 1

```
</> Java
new User.Builder()
```

Creates a **Builder** object.

```
Builder
name = null
age = 0
city = null
```

Step 2

```
</> Java
.name("Supraja")
```

Builder becomes:

```
Builder
name = "Supraja"
age = 0
city = null
```

Step 3

```
</> Java
.age(25)
```

```
Builder
name = "Supraja"
age = 25
city = null
```

Step 4

```
</> Java
.city("Hyderabad")
```

```
Builder
name = "Supraja"
age = 25
city = "Hyderabad"
```

Step 5

```
</> Java
.build()
```

Now Builder says:

I have collected all the values. Let me create the actual User.

```
</> Java
return new User(this);
```

So finally:

```
User
name = "Supraja"
age = 25
city = "Hyderabad"
```

8. Why is build() necessary?

This is a very common interview question.

Why not simply do:

```
</> Java
User user = new User.Builder()
    .name("Supraja")
    .age(25);
```

Because at this point you're still dealing with a **Builder**, not a **User**.**9. Why do we often make the actual object immutable?**

You may see:

```
</> Java
```

```
private final String name;
private final int age;
private final String email;
```

and no setters.

For example:

```
// Java
class User {

    private final String name;
    private final int age;

    private User(Builder builder) {
        this.name = builder.name;
        this.age = builder.age;
    }
}
```

Once:

```
// Java
User user = builder.build();
```

is executed, the User's state cannot be changed.

That's useful because immutable objects are:

- easier to reason about
- safer to share
- naturally thread-safe when designed correctly
- less prone to accidental modification

11. When should I use Builder?

Use Builder when:

Many parameters

```
// Java
User(name, age, email, phone, address, city, country...)
```

Many optional parameters

For example:

```
Car
├─ engine      required
├─ wheels     required
├─ sunroof    optional
├─ GPS        optional
├─ leatherSeat optional
├─ heatedSeat optional
└─ spoiler    optional
```

Builder makes this clean:

```
// Java
Car car = new Car.Builder()
    .engine("V8")
    .sunroof(true)
    .gps(true)
    .leatherSeats(true)
    .build();
```

Instead of:

```
// Java
new Car("V8", 4, true, true, false, true, false);
```

12. Real-world Java example

You've probably already seen Builder pattern without realizing it.

For example, Java/Spring APIs often use things like:

```
// Java
PageRequest.of(page, size)
```

or libraries may provide APIs such as:

```
// Java
HttpRequest.newBuilder()
    .uri(...)
    .header(...)
    .timeout(...)
    .build();
```

The second one is a classic Builder-style API.

The pattern is:

```
newBuilder()
```

```

↓
configure options
↓
build()
↓
final object

```

Builder is a creational design pattern used to construct complex objects step by step. It is especially useful when an object has many optional parameters, because it avoids telescoping constructors and makes object creation more readable. The builder collects the required values and the build() method creates the final object, often performing validation along the way

Why is builder class static? If builder is not static, we need to create user obj which doesn't make sense to be created before builder object 😊

Yes — this is confusing at first. Let's focus **only on why Builder is static**.

Suppose we have:

```

<> Java
class User {
    private String name;
    private int age;

    static class Builder {
        private String name;
        private int age;
    }
}

```

The **Builder** is **static** because a **Builder** does not need an existing **User** object.

Without static

If you write:

```

<> Java
class User {
    class Builder {
    }
}

```

Builder becomes an **inner class**, which means every **Builder** object is implicitly associated with a particular **User** object.

So you'd have to do:

```

<> Java
User user = new User();

User.Builder builder = user.new Builder();

```

Notice ~~the problem~~:

```

User object
↓
needed to create
↓
Builder object

```

~~But what are~~ we trying to do with the Builder?

Create a User!

So requiring a **User** before creating the **Builder** doesn't make logical sense.

With static

```

<> Java
class User {
    static class Builder {
    }
}

```

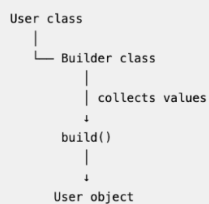
Now **Builder** belongs to the **User class**, rather than to a particular User object.

So we can simply do:

```
Java
User.Builder builder = new User.Builder();
```

No **User** object is needed.

The flow becomes:



That's exactly what we want.

Question 2: return new User(this); -> deep dive:

2. Typecasting looks completely different

Typecasting would look like:

```
Java
User u = (User) someObject;
```

Here:

```
Java
(User)
```

is the cast.

It means:

"Treat the object referenced by `someObject` as a `User`."

But `this` doesn't do that.

```
Java
new User(this)
```

has no cast at all.